السلام عليكم ورحمة الله وبركاته.. وبعد،

بين يديك ملخص شامل ومنظم لمادة CPCS-361 ، تم إعداده بعناية ليكون عوناً لك في فهم المادة .وقبل البدء، أضع بين يديك ميثاقاً للأمانة العلمية والتعاون:

⚠️ ميثاق الأمانة والمسؤولية:

هذا العمل هو جهد طالب، أضعه بين يديك بنية النفع والفائدة؛ لذا:

- **لا أسامح ولا أحلل استخدامه في الغش أو أي عمل ينافي الأخلاق الإسلامية.**
- **يُمنع منعاً باتاً تداوله كمصدر تجاري للباعة أو استخدامه في أدوات الذكاء الاصطناعي لغرض التحايل او الغش.**
- **الأمانة العلمية تقتضي منك استخدامه كمرجع للفهم والمراجعة، لا كبديل عن الضمير والاجتهاد الشخصي.**

📣 التواصل معي:

الكمال لله وحده، لذا يسعدني استقبال ملاحظاتكم، تعديلاتكم، أو إضافاتكم عبر:

- **الرقم المباشر: 0530717788**
- **الموقع الشامل: https://kau-one.web.app/contact/**

توجيه إيماني:

قبل أن تبدأ، استعذ بالله من الشيطان الرجيم، وجدد نيتك بأن يكون هذا الوقت في سبيل طلب العلم النافع الذي يرفع قدرك في الدنيا والآخرة. قال رسول الله ﷺ: "مَنْ سَلَكَ طَرِيقًا يَلْتَمِسُ فِيهِ عِلْمًا سَهَّلَ اللَّهُ لَهُ بِهِ طَرِيقًا إِلَى الْجَنَّةِ" (رواه مسلم).

إعداد الطالب: إلياس حسان بابور الدفعة: 24 — تم اخذ الماده و عمل الملخص على السلايدات سنه م2026

Chapter 2

1. Introduction & What is an Operating System? (مقدمة وما هو نظام التشغيل؟)

- **Definition:** A program that acts as an intermediary (وسيط) between a user and the computer hardware.
- **Goals of the OS:**
    1. Execute user programs and make solving problems easier.
    2. Make the computer system convenient (مريح وسهل) to use.
    3. Use computer hardware in an efficient manner (بكفاءة).

2. Computer System Structure (بنية نظام الحاسب)

Divided into 4 components:

1. **Hardware (المكونات المادية):** CPU, memory, I/O devices (provides basic computing resources).
2. **Operating System:** Controls and coordinates hardware use among applications and users.
3. **Application Programs:** Word processors, compilers, browsers, video games (define how resources are used to solve user problems).
4. **Users:** People, machines, or other computers.

3. The Two Viewpoints of an OS (وجهتي نظر لنظام التشغيل)

A. User View (نظرة المستخدم) - depends on the device:

- **Single-user (PCs):** Goal is to maximize the work/play of one user. Designed for ease of use and performance. The user monopolizes (يحتكر) resources.
- **Multi-user (Mainframes/Servers):** Users share resources. Designed to maximize resource utilization—ensuring CPU, memory, and I/O are shared efficiently and fairly.
- **Handheld (Smartphones/Tablets):** Touch screen interface. Designed for usability and battery life (because resources are poor/limited).
- **Embedded (Home devices/Automobiles):** Numeric keypads/indicator lights. Designed to run without user intervention (بدون تدخل المستخدم).

B. System View (نظرة النظام):

- **Resource Allocator (موزع موارد):** Manages resources and decides between conflicting requests fairly and efficiently.
- **Control Program (برنامج تحكم):** Controls program execution to prevent errors and improper use.

4. Defining the OS (تعريف نظام التشغيل)

- There is no universally accepted definition.
- **The Kernel (النواة):** The common definition is that the OS is the one program running at all times on the computer.
- **Other Programs:**
    - *System Programs:* Associated with the OS but not part of the kernel.
    - *Application Programs:* All other programs.
- **Mobile OS (Android/iOS):** They include the core kernel *plus* **Middleware** (إطار عمل برمجي يوفر خدمات إضافية مثل قواعد البيانات والرسوميات).

5. Computer System Organization & Startup (تنظيم النظام والإقلاع)

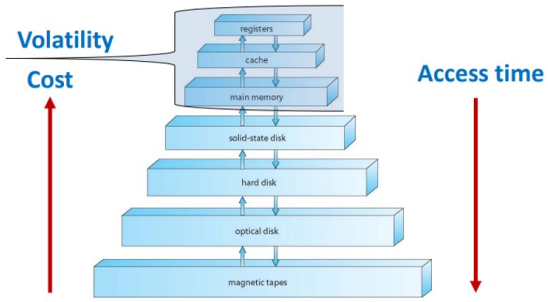
- **Organization:** CPU and device controllers connect through a common bus to shared memory. They can execute in parallel, competing for memory cycles. A memory controller synchronizes this.
- **Computer Startup:** A **Bootstrap program (برنامج الإقلاع)** is loaded at power-up.
    - Stored in ROM or EPROM (known as **Firmware**).
    - It initializes the system, loads the OS kernel into memory, and starts execution.

6. I/O Structure & Interrupts (الإدخال/الإخراج والمقاطعات)

- **Interrupts (المقاطعات):** An OS is "interrupt-driven". A device controller informs the CPU it finished a task by causing an interrupt.
    - The interrupt transfers control to an *interrupt service routine* via an *interrupt vector* (which holds the addresses of all routines).
    - Architecture must save the address of the interrupted instruction so it can return to it later.
- **Software Interrupts (Trap or Exception):** An interrupt generated by software due to:
    - A software error (e.g., dividing a number by zero).
    - A specific request for an OS service.
- **I/O Methods:**
    - *Method 1 (Wait):* CPU is idle (wait instruction/loop) until I/O finishes. Only one I/O request runs at a time.
    - *Method 2 (System Call):* Control returns to the user program immediately without waiting. The OS uses a **Device-status table** to keep track of devices and their states.

7. Storage Structure & Hierarchy (هيكلية التخزين)

- **Main Memory (RAM):** The only large storage the CPU can access directly. It is random access and volatile (يفقد البيانات عند انقطاع الكهرباء).
- **Secondary Storage:** Non-volatile extension of main memory.
    - *Hard Disks:* Rigid platters covered in magnetic material. Logically divided into tracks and sectors.
    - *Solid-State Disks (SSD):* Faster than hard disks, non-volatile, becoming very popular.
- **Storage Hierarchy:** Organized by **Speed, Cost, and Volatility**.



- **Caching (التخزين المؤقت):** Information is temporarily copied from slower storage to faster storage (Cache). If data is not in the cache, it is copied there for fast access. Cache size and replacement policy are major design problems.

8. Direct Memory Access (DMA - الوصول المباشر للذاكرة)

- Used for high-speed I/O devices (like disks or network cards).
- Device controller transfers whole blocks of data directly to main memory *without* bothering the CPU.
- **Advantage:** Generates only **one interrupt per block**, instead of one interrupt per byte.

9. Computer-System Architecture (بنيات الأنظمة)

• Most systems use a single general-purpose processor

- **Multiprocessors (Parallel / Tightly-coupled systems):** Have multiple CPUs.
    - *Advantages:* 1. Increased throughput. 2. Economy of scale. 3. Increased reliability .
    - *Types:*
        - **Asymmetric:** Each processor has a specific assigned task.
        - **Symmetric (SMP):** All processors perform all tasks.
- **Clustered Systems (الأنظمة العنقودية):** Multiple systems working together, usually sharing a Storage-Area Network (SAN).
    - Provides high-availability (survives failures).
    - *Asymmetric clustering:* One machine runs, the other is in "hot-standby" mode.
    - *Symmetric clustering:* Multiple nodes run and monitor each other.
    - Some have a **Distributed Lock Manager (DLM)** to prevent data conflicts.

10. Operating-System Structure - *[Detailed Section]*

- **Multiprogramming (Batch System - البرمجة المتعددة):**
    - Needed for efficiency so the CPU/IO devices are never idle.
    - OS keeps a subset of jobs in memory.
    - Uses **job scheduling** to pick one job. When it waits (e.g., for I/O), the OS instantly switches to another job.
- **Timesharing (Multitasking - تعدد المهام):**
    - CPU switches jobs so frequently that users can interact with each program (Interactive computing).
    - **Response time should be < 1 second.**
    - Each user has a program in memory called a **Process**.
    - **CPU Scheduling:** Used if several jobs are ready to run at the same time.
    - **Swapping (التبديل):** If processes don't fit in memory, they are moved in and out of the disk to run.
    - **Virtual Memory (الذاكرة الافتراضية):** Allows execution of processes that are not completely in memory (gives the illusion of infinite memory).

11. Operating-System Operations (عمليات النظام)

- **Interrupt Driven:** (As mentioned, driven by hardware or software/trap/exception like division by zero, OS requests, infinite loops, or processes modifying each other).
- **Dual-Mode Operation (وضع التشغيل المزدوج):** Protects the OS and system components using a hardware "Mode bit".
    - **User Mode:** Running normal code.
    - **Kernel Mode:** Running OS code.
    - *Privileged instructions* can ONLY be executed in Kernel mode.
    - *System calls* change the mode to Kernel, and reset it to User mode when returning.
- **Timer (المؤقت):** Prevents infinite loops or processes hogging (احتكار) resources.
    - Set to interrupt the computer after a time period.
    - A counter decrements by the physical clock. When it hits zero, it generates an interrupt.
    - OS sets this counter (Privileged instruction) before scheduling a process to regain control if the program runs too long.

12. Process Management (إدارة العمليات)

- **What is a Process?** A program in execution. It is an *active* entity (كيان نشط), whereas a program on a disk is a *passive* entity (كيان خامل). It is the basic unit of work in the system.
- **Resources Needed:** CPU, memory, I/O devices, files, and initialization data. When a process terminates, the OS must reclaim (يستعيد) any reusable resources.
- **Threads (المسارات أو الخيوط):**
    1. *Single-threaded:* Has one "program counter" (عداد البرنامج). It executes instructions sequentially, one at a time.
    2. *Multi-threaded:* Has one program counter *per thread*, allowing multiple tasks within the same process.
- **Concurrency (التزامن):** The system multiplexes (يوزع وقت) the CPUs among many running processes/threads to make them seem like they are running at the same time.
- **OS Activities in Process Management:**
    1. Creating and deleting user and system processes.
    2. Suspending (إيقاف مؤقت) and resuming processes.
    3. Providing mechanisms for **Process Synchronization** (مزامنة العمليات لتجنب تداخل البيانات).
    4. Providing mechanisms for **Process Communication** (تبادل البيانات بين العمليات).
    5. Providing mechanisms for **Deadlock Handling**
        - حالة تتوقف فيها عمليتان أو أكثر إلى الأبد، لأن كل عملية تنتظر المورد الذي تملكه العملية الأخرى. النظام يوفر طرقاً للتعامل مع هذه المشكلة.

13. Memory Management (إدارة الذاكرة)

- **The Rule:** To execute a program, all (or part) of its instructions and data *must* be in main memory.
- **Goal:** Optimizing CPU utilization and computer response to users.
- **OS Activities:**
    1. Keeping track of which parts of memory are currently being used and by whom.
    2. Deciding which processes (or parts) to move into and out of memory.
    3. Allocating (تخصيص) and deallocating (تحرير) memory space as needed.

14. Storage Management (إدارة التخزين)

- **Logical Storage (File-System Management):**
    - The OS abstracts physical properties into a logical unit called a **File** (ملف), usually organized into **Directories** (مجلدات).
    - **OS Activities:** Creating/deleting files, mapping files to secondary storage, and backing up files to stable/non-volatile media. Provides access control (who can access what).
- **Mass Storage (التخزين الشامل):**
    - Disks store data that doesn't fit in main memory or needs to be kept long-term. The entire speed of the computer hinges on the disk subsystem.
    - **OS Activities:** Free-space management, storage allocation, and **Disk Scheduling** (جدولة طلبات القراءة والكتابة على القرص).

- **Tertiary Storage:** Optical storage or magnetic tape. Can be WORM (Write-Once, Read-Many) or RW (Read-Write).
- **Data Migration & Cache Coherency (هجرة البيانات وتطابق المخبأ):**
    - Data moves from Disk -> RAM -> Cache -> Registers.
    - In a multiprocessor system, the OS/hardware must provide **Cache Coherency** (تطابق الذاكرة المخبئية): ensuring that if CPU 1 modifies data, CPU 2 immediately sees the updated value, not the old one.

15. I/O Management (إدارة الإدخال والإخراج)

- **Purpose:** To hide the exact peculiarities (تفاصيل معقدة) of hardware devices from the user.
- **I/O Subsystem Responsibilities:**
    1. **Buffering (الجدولة في الذاكرة المؤقتة):** Storing data temporarily while it is being transferred.
    2. **Caching (التخزين المخبئي):** Storing parts of data in faster storage for performance.
    3. **Spooling:** Overlapping the output of one job with the input of other jobs (e.g., sending multiple print jobs to a printer queue).
    4. Managing general device-driver interfaces and drivers for specific hardware.

16. Kernel Data Structures (هياكل بيانات النواة)

The OS uses standard programming data structures:

- **Linked Lists:** Singly (اتجاه واحد), Doubly (اتجاهين), Circular (دائرية).
- **Binary Search Tree:** Search performance is O(n), but if it's a *balanced* tree, it is O(lg n).
- **Hash Map:** Uses a hash function to map a key to a value.
- **Bitmap:** A string of binary digits (0s and 1s) representing the status of items (e.g., which blocks on a disk are free or used).

17. Computing Environments (بيئات الحوسبة)

- **Traditional :** Stand-alone general purpose machines But blurred as most systems interconnect with others (i.e., the Internet) - Portals provide web access to internal systems - Network computers (thin clients) are like Web terminals
  - Mobile computers interconnect via wireless networks - Networking becoming ubiquitous – even home systems use firewalls to protect home computers from Internet attacks
- **Mobile:** Smartphones/Tablets. Extra OS features like GPS/gyroscope. Connectivity via IEEE 802.11 (Wi-Fi) or cellular. Leaders: Apple iOS, Google Android.
- **Distributed (الأنظمة الموزعة):** Separate systems networked together (LAN, WAN, MAN, PAN) communicating via TCP/IP. The Network OS provides the illusion of a single system.
- **Client-Server:** Compute-server (client requests services like a DB) and File-server (client stores/retrieves files).

- **Peer-to-Peer (P2P - الند للند):** No distinction between client and server. All nodes are "peers" acting as both (e.g., Skype, Napster). Nodes broadcast requests or use a central lookup.
- **Virtualization (الأنظمة الوهمية):**
    - *Emulation (المحاكاة):* Slow. Used when the source CPU type is different from the target (e.g., running Mac software on an Intel Windows PC).
    - *Virtualization:* Natively compiled. Running guest OSes on a host OS using a **VMM (Virtual Machine Manager)** like VMware.
- **Cloud Computing (الحوسبة السحابية):** Delivers computing, storage, or apps across a network (uses virtualization).
    - *Types:* Public, Private, Hybrid.
    - *Models:* **SaaS** (Software as a Service - e.g., Word processor online), **PaaS** (Platform as a Service - e.g., DB server ready for developers), **IaaS** (Infrastructure as a Service - e.g., storage for backup).
- **Real-Time Embedded Systems:** Have well-defined, strict **fixed time constraints** (قيود زمنية صارمة). Processing *must* be done within the constraint, or the system fails.
- **Open-Source OSs:** Source code is available (e.g., Linux). Counter to DRM (Digital Rights Management). Often licensed under GPL (Copyleft).

18. Operating System Services (خدمات نظام التشغيل)

- **User-helpful services:** UI, Program execution, I/O operations, File-system manipulation, Communications (via *Shared Memory* or *Message Passing*), Error detection (CPU, memory, or software errors).
- **System-efficiency services:** Resource allocation (CPU, memory, files), Accounting (tracking which users use what resources), Protection (controlling access) and Security (defending from outsiders/authentication).

19. User Interfaces (واجهات المستخدم)

- **CLI (Command Line):** Direct command entry. Fetches a command and executes it (e.g., Bourne Shell).
- **GUI (Graphical):** Desktop metaphor (icons, mouse, folders). Invented at Xerox PARC. Mac uses Aqua GUI over UNIX kernel. Windows uses GUI + CLI.
- **Touch Screen:** Gestures, virtual keyboard, voice.

20. System Calls (نداءات النظام)

Definition

- A **system call** is a programming interface to the services provided by the **Operating System (OS)**.
- System calls are typically written/implemented in **C or C++**.
- Programs usually access system calls through a **high-level API** rather than calling them directly
- Common APIs:
    - **Win32 API** → Windows
    - **POSIX API** → UNIX, Linux, and macOS
    - **Java API** → Java Virtual Machine (JVM)
- The API hides many OS details from the programmer.

System Call Implementation

- Each system call typically has a **unique number**.
- The **system-call interface** maintains a table indexed by these numbers.
- The interface:
    - Receives the requested system call.
    - Invokes the corresponding system call in the OS kernel.
    - Returns the status and any return values.

- The programmer does **not** need to know how the system call is implemented.
- The programmer only needs to:
    - Follow the API.
      Understand what the OS will do as a result of the call.
- The system-call interface is managed by the **run-time support library**, which contains functions provided through libraries included with the compiler.

Parameter Passing (تمرير المعطيات للنظام)

Sometimes more information is needed than just the identity of the system call. There are **3 general methods**:

1. **Registers (المسجلات)**
    - Parameters are passed directly in registers.
    - Simplest method.
    - Problem: there may be more parameters than available registers.

2. **Block / Table (كتلة أو جدول في الذاكرة)**
    - Parameters are stored in a block/table in memory.
    - The address of the block is passed in a register.
    - Used by **Linux and Solaris**.

3. **Stack (المكدس)**
    - The program pushes parameters onto the stack.
    - The OS pops them from the stack.

Important:

- Block and stack methods do **not limit the number or length of parameters** being passed.

Types of System Calls

1. Process Control

- Create and terminate processes.
- End or abort a process.
- Load and execute a program.
- Get/set process attributes.
- Wait for a specified time.
- Wait for an event / signal an event.
- Allocate and free memory.
- Dump memory when an error occurs.
- Debugging and single-step execution.
- Locks for managing access to shared data between processes.

2. File Management

- Create and delete files.
- Open and close files.
- Read, write, and reposition files.
- Get and set file attributes.

3. Device Management

- Request and release devices.
- Read, write, and reposition.
- Get and set device attributes.
- Logically attach or detach devices.

4. Information Maintenance

- Get or set time and date.
- Get or set system data.
- Get and set process, file, or device attributes.

5. Communications

- Create and delete communication connections.
- Send and receive messages.
- Communication can occur between processes or between client and server.
- Shared-memory model: create and access shared memory regions.
- Transfer status information.
- Attach and detach remote devices.

6. Protection

- Control access to resources.
- Get and set permissions.
- Allow or deny user access.

	Windows	Unix

Process Control	`CreateProcess()`
	`ExitProcess()`
	`WaitForSingleObject()`
		`fork()`
		`exit()`
		`wait()`

File Manipulation	`CreateFile()`
	`ReadFile()`
	`WriteFile()`
	`CloseHandle()`
		`open()`
		`read()`
		`write()`
		`close()`

Device Manipulation	`SetConsoleMode()`
	`ReadConsole()`
	`WriteConsole()`
		`ioctl()`
		`read()`
		`write()`

Information Maintenance	`GetCurrentProcessID()`
	`SetTimer()`
	`Sleep()`
		`getpid()`
		`alarm()`
		`sleep()`

Communication	`CreatePipe()`
	`CreateFileMapping()`
	`MapViewOfFile()`
		`pipe()`
		`shmget()`
		`mmap()`

Protection	`SetFileSecurity()`
	`InitlializeSecurityDescriptor()`
	`SetSecurityDescriptorGroup()`
		`chmod()`
		`umask()`
		`chown()`

System Call Example

- A C program may call the library function `printf()`.
- The library function can then invoke the `write()` system call.
- Therefore:

FreeBSD Example

- FreeBSD is a **UNIX variant** and supports **multitasking**.
- When a user logs in, the user's selected shell is started.
- The shell:
  - Uses `fork()` to create a new process.
  - Uses `exec()` to load the requested program into that process.
    The shell may wait for the process to terminate or continue accepting commands.
- Process exit codes:
  - **0** → no error
  - **> 0** → error code

MS-DOS Example

- MS-DOS is **single-tasking**.
- It uses a **single memory space**.
- No new process is created when running a program.
- The program is loaded into memory, **overwriting everything except the kernel**.
- When the program exits, the **shell is reloaded**.

21. OS Design & Implementation(تصميم وتنفيذ نظام التشغيل)

- OS Design

- The internal structure of different operating systems can vary widely.
- It depends on:
    - The hardware being used.
    - The type of system.
- OS design begins by defining:
    - **User goals**
    - **System goals**

- Implementation Languages

- Early operating systems were written mainly in **Assembly language**.
- Later, languages such as **Algol and PL/1** were used.
- Modern operating systems mainly use **C and C++**.

Advantages of C/C++

- Faster to write.
- More compact.
- Easier to understand.
- Easier to debug.
- Easier to **port to different hardware**.

Disadvantages

- Reduced performance compared with Assembly.
- May require more storage.

Performance Improvement

- To achieve better performance, **bottleneck routines** can be replaced with Assembly-language versions.

22. Debugging(معالجة الأخطاء)

- Definition

Debugging = finding and fixing errors (bugs).

- Error Information

Operating systems can generate **log files** containing information about errors.

- **Application failure → Core dump**
  - Captures the memory of the failed process.

- **OS/kernel failure → Crash dump**
  - Contains information about kernel memory.

- Debugging Tools

- Example: **DTrace** in Solaris.
- DTrace uses probes that activate when code is executed.
- The probes capture state information and send it to consumers of the probes.

- Performance Tuning

- Performance tuning means improving system performance by **finding and removing bottlenecks**.
- The OS should provide ways to measure and display system behavior.
- Examples:

  - `top` in Linux/Unix
  - **Task Manager** in Windows

23. Operating System Generation (SYSGEN)(تكوين نظام التشغيل)

- Definition

- Operating systems are designed to run on many different classes of machines.
- Therefore, the OS must be **configured for each specific computer**.
- This process is called **System Generation (SYSGEN)**.

- SYSGEN Program

The SYSGEN program obtains information about the specific hardware configuration, such as:

- **CPU:** What CPU is being used?
- **CPU options:** Are extended instruction sets or floating-point arithmetic available?
- **Memory:** How much memory is available?
- **Devices:** What devices are available and how are they addressed?
- **OS options:** What operating-system options are required?

24. System Boot(إقلاع النظام)

- Booting

- **Booting** means starting a computer by **loading the OS kernel**.

- Bootstrap Program

- A **bootstrap program** is a small piece of code stored in **ROM**.

- Its job is to:
    1. Locate the OS kernel.
    2. Load the kernel into memory.
    3. Start execution of the kernel.

- Two-Step Boot Process

- Sometimes booting happens in **two steps**:

    1. A small bootstrap loader loads a larger loader.
    2. The larger loader loads the OS kernel.

Main idea: The OS must be made available to the hardware so that the hardware can start the operating system.

Chapter 3

Introduction — Process (العملية)

What is a Process?

- A **process** is a **program in execution**.
- It is a **unit of work** within the system.
- **Program = passive entity**
    - A program is simply a set of instructions stored on disk.
- **Process = active entity**
    - A process is a program that is currently being executed.

Process Terminology

- **Batch System** → executes **jobs**.
- **Time-Shared System** → has **user programs or tasks**.
- Traditionally, each process contains **one thread**.
- Modern operating systems support **multithreaded processes**, meaning one process can contain multiple threads.

Process Components

A process contains more than just the program code. It consists of several sections:

1. Text Section (Code)

- Contains the **actual program code/instructions**.
- It represents the instructions that the process executes.

2. Program Counter (PC)

- Keeps track of the **current activity of the process**.
- Contains the address/location of the **next instruction to be executed**.

3. CPU Registers

- Store information needed by the CPU while executing the process.
- Their contents are saved as part of the process's execution state.

4. Stack

- Contains temporary data used during function calls.
- Examples:
    - Function parameters
    - Return addresses
    - Local/temporary variables

5. Data Section

- Contains the process's **global variables**.

6. Heap

- Contains memory that is **dynamically allocated during runtime**.

- Process Control Block (PCB)(كتلة التحكم بالعملية)

- Each process is represented by its own **Process Control Block (PCB)**.
- PCB is sometimes called a **Task Control Block (TCB)**.
- The PCB contains **all information required for the successful execution and management of a process**.
- The OS uses the PCB to keep track of each process.

PCB Contains:

1. Process State

- Indicates the current state of the process.
- Examples:
    - **Running**
    - **Waiting**
    - etc.

2. Program Counter

- Contains the location/address of the **next instruction to execute**.

3. CPU Registers

- Contains the contents of all **process-related CPU registers**
- This information is important when the OS needs to stop and later resume a process.

4. I/O Status Information

- Contains information about:
    - I/O devices allocated to the process.
    - List of open files.

5. CPU Scheduling Information

- Contains information used by the CPU scheduler, such as:
    - Process priority.
    - Pointers to scheduling queues.

6. Memory Management Information

- Contains information about the **memory allocated to the process**.

7. Accounting Information

- Contains information such as:

    - CPU time used.
    - Clock time elapsed.
    - Time limits.

Important idea:

The **PCB is the OS's record/card for a process**.

It tells the OS things like:

What state is the process in?
Where did it stop?
What are its CPU registers?
What memory does it use?
What files/devices does it have?
What is its priority?
How much CPU time has it used?

Process in Linux

In Linux, a process is represented by a C structure called: `task_struct`

It contains information about the process Important fields:

`task_struct` field	Meaning
`pid`	Process ID
`state`	Process state
`time_slice`	Scheduling information
`parent`	Parent process
`children`	Child processes
`files`	Open files
`mm`	Process address space

- Operations on Processes(العمليات التي يمكن إجراؤها على العملية)

There are **TWO main operations** that can be performed on a process:

1. **Process Creation** (إنشاء العملية)
   Process Termination (إنهاء العملية)

1. Process Creation

Basic Concept

- Every process is generally identified and managed using a **Process Identifier (PID)**.
- A **parent process** can create one or more **child processes**.
- A child process can also create other processes.
- This creates a **tree of processes** called a **process tree**.

Process Creation has two main aspects:

1. **Resource Sharing**
2. **Process Execution**

A. Resource Sharing

When a parent creates a child, there are **3 possible ways** resources can be shared:

1. Parent and children share all resources

- The parent and child processes have access to all the parent's resources.

2. Children share a subset of parent's resources

- The child receives only **some** of the resources available to the parent.

3. Parent and child share no resources

- The child has its own resources and does not share the parent's resources.

basically:

Resource Sharing = All / Some / None

B. Process Execution

There are two possibilities for how the parent and child execute:

1. Parent and children execute concurrently

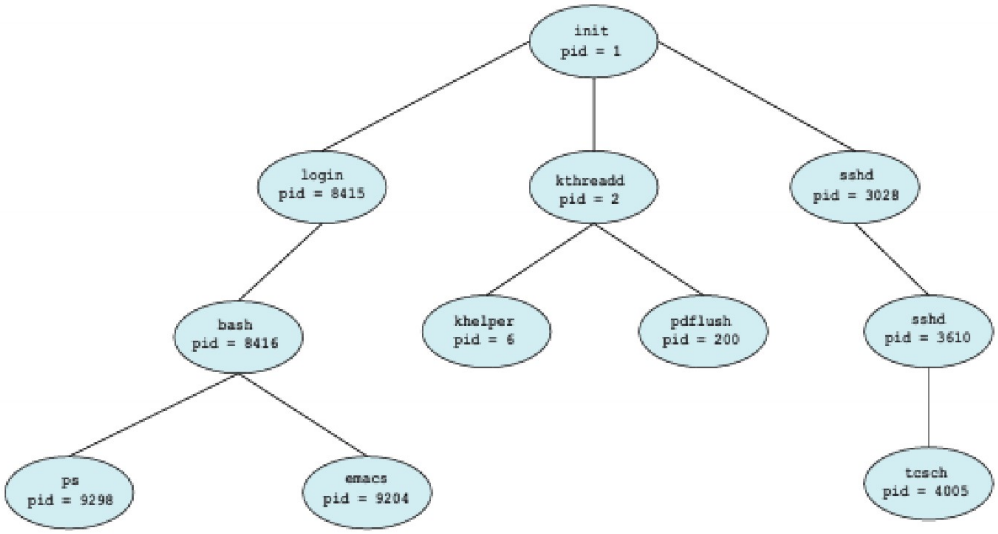
- The parent continues executing while the child also executes.

2. Parent waits until children terminate

- The parent stops/waits until the child process finishes.

Process Tree in Linux

Processes form a hierarchical **tree structure**.



Important idea

- Each process can have:
    - A **parent**
    - Zero or more **children**
- This creates a hierarchical process structure.

1.Process Creation in UNIX/Linux

Address Space

When a process creates a child:

- The child may initially have an **address space that is a duplicate of the parent**.
- The child can then have a **new program loaded into it**.

Important UNIX System Calls

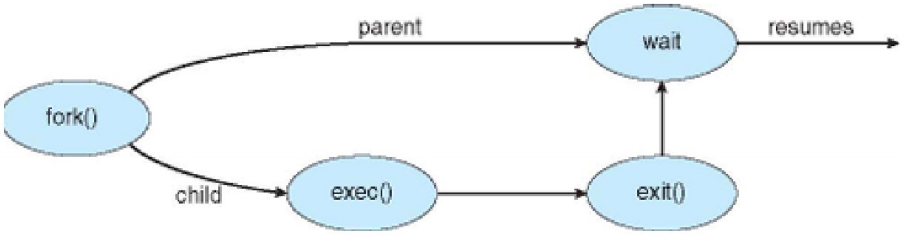
`fork()`

- Creates a **new process**.
- The new process is the **child process**.

`exec()`

- Used after `fork()`.
- Replaces the process's memory/address space with a **new program**.

Typical sequence:



2.Process Termination

Normal Termination

- A process executes its **last statement**.
- It then asks the OS to delete/terminate it.
- The `exit()` system call is used to terminate the process.

What happens during termination?

1. Process finishes its execution.
2. Process calls `exit()`.
3. Status information can be returned to the parent using `wait()`.
4. The OS **deallocates/releases the process's resources**.

`wait()` System Call

- A parent process can use `wait()` to wait for a child process to terminate.
- `wait()` returns:
    - Status information.
    - The **PID of the terminated child**.

Example:

pid = wait(&status);

Where:

- `pid` → PID of the terminated child.
  `status` → termination/status information.

Important:

`exit()` → child terminates
`wait()` → parent waits for child and receives its status

- Termination by Parent Process

A parent process may terminate a child using the **`abort()` system call**.

Why might a parent terminate a child?

There are several reasons:

1. Child exceeded allocated resources

- The child used more resources than it was allowed to use.

2. Child's task is no longer required

- The work assigned to the child is no longer needed.

3. Parent is terminating

- Some operating systems do not allow a child to continue if its parent terminates.

- Cascading Termination

- If a process terminates, all of its children may also need to be terminated.
- This is called **cascading termination**.

Inter-Process Communication (IPC)

1. IPC — Introduction

Processes in a system can be divided into **TWO types**:

1. Independent Processes

- Do **not need to communicate** with other processes.
- Their execution does not depend on other processes.

2. Cooperating Processes

- **Need to communicate** with other processes to complete their execution.
- IPC provides mechanisms that allow these processes to communicate and coordinate.

Reasons for Process Cooperation

Processes may cooperate for several reasons:

1. **Information Sharing**
    - Processes need to share information/data.
2. **Computation Speedup**
    - A task can be divided among multiple processes to speed up computation.
3. **Modularity**
    - A system can be divided into separate cooperating processes/modules.
4. **Convenience**
    - Multiple processes can perform different tasks at the same time.

2. IPC Models

There are **TWO main models** of IPC:

1. **Shared Memory**

2. **Message Passing**

3. Shared Memory

Definition

- A region/area of memory is **shared among processes** that need to communicate.
- Processes communicate by reading and writing to this shared memory.

Important Point

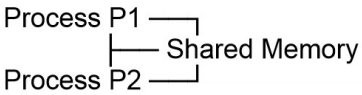
The communication is mainly under the **control of the user processes**, rather than being controlled directly by the OS for every communication.

Main Problem

The major issue is **synchronization**.

Processes must coordinate their actions when accessing shared memory to avoid conflicts.

For example:



If both processes access/change the same data at the same time, problems can occur.

---

4. Producer–Consumer Problem

The **Producer–Consumer Problem** is a common example of cooperating processes.

Producer

- Produces information/data.
- Places the information into a buffer.

Consumer

- Takes/consumes information from the buffer.

- Buffer Types

1. Unbounded Buffer

- There is **no practical limit** on the size of the buffer.
- The producer can continue producing without worrying about the buffer becoming full.

2. Bounded Buffer

- The buffer has a **fixed size**.
- If the buffer becomes full, the producer must wait.

5. Message Passing

Definition

Message passing provides a mechanism for processes to:

- Communicate with each other.
- Synchronize their actions.

Main Idea

Processes communicate by **sending and receiving messages**, rather than using shared variables.

Two Main Operations

send(message)
receive(message)

The message size can be:

- **Fixed**
- **Variable**

6. Communication Between Two Processes

If processes **P and Q** want to communicate, they must:

1. **Establish a communication link** between them.
2. **Exchange messages** using `send()` and `receive()`.

7. Message-Passing Implementation Issues

When implementing a communication link, several questions must be answered:

- How are communication links established?
- Can one link be associated with more than two processes?
- How many links can exist between each pair of communicating processes?
- What is the capacity of the link?
- Is the message size **fixed or variable**?
- Is the link **unidirectional or bidirectional**?

8. Communication Link Implementation

Communication links can be considered in terms of **physical** and **logical** implementation.

A. Physical Implementation

Possible physical mechanisms include:

- **Shared memory**
- **Hardware bus**
- **Network**

B. Logical Implementation

The logical characteristics include:

- **Direct or indirect communication**
- **Synchronous or asynchronous communication**
- **Automatic or explicit buffering**

9. Direct Communication

Definition: Processes must explicitly name the process they want to communicate with.

Operations

send(P, message)

- Send a message to process **P**.

receive(Q, message)

- Receive a message from process **Q**.

Properties

- Communication links are established **automatically**.
- A link is associated with exactly **one pair of communicating processes**.
- Between each pair of processes, there is **exactly one link**.
- The link may be **unidirectional**, but is usually **bidirectional**.

10. Indirect Communication

Definition: Processes communicate through an intermediate **mailbox (port)**.

Instead of sending directly to another process, a process sends the message to a mailbox.

P1 → Mailbox A ← P2

Mailbox Operations

Three basic operations:

1. **Create a mailbox**
2. **Send/receive messages through the mailbox**
3. **Destroy the mailbox**

Primitives

send(A, message) : Send a message to mailbox **A**.

receive(A, message): Receive a message from mailbox **A**.

Basically :

Indirect = Process → Mailbox → Process

11. Mailbox Sharing Problem

Suppose:

- P1, P2, and P3 share mailbox **A**.
- P1 sends a message.
- Both P2 and P3 can receive.

Question:

Who gets the message?

There are several possible solutions.

Solution 1

- Allow a communication link to be associated with **at most two processes**.

Solution 2

- Allow only **one process at a time** to execute a `receive()` operation.

Solution 3

- Let the system **arbitrarily select the receiver**.
- The sender is then notified about which process received the message.

12. Blocking Message Passing

Message passing can be **blocking** or **non-blocking**.

Blocking = Synchronous : The process may have to wait.

Blocking Send : send(message)

- The sender is **blocked until the message is received**.

Blocking Receive: receive(message)

- The receiver is **blocked until a message is available**.

13. Non-Blocking Message Passing

Non-blocking = Asynchronous

The process does **not have to wait**.

Non-blocking Send

- The sender sends the message and **continues execution immediately**.

Non-blocking Receive

The receiver gets either:

- A **valid message**, or
- A **null message** if no message is available.

15. Producer–Consumer with Message Passing

The Producer–Consumer problem can also be implemented using **message passing**.

Buffering

Messages can be stored in a **queue attached to the communication link**.

There are **3 buffering implementations**:

1. Zero Capacity

- The link has **no messages queued**.
- The sender must wait until the receiver is ready.

This is called a **rendezvous**.

Zero capacity = Sender waits for receiver

2. Bounded Capacity

- The link has a **finite buffer** that can hold n messages.
- If the buffer is full, the sender must wait.

Bounded = finite capacity

3. Unbounded Capacity

- The link has a theoretically **infinite-length queue**.
- The sender **never has to wait because the buffer is full**.

Unbounded = sender never waits because of buffer capacity

Chapter 4

Process Management and CPU Scheduling.

1. Fundamental Process Concepts

Difference Between Program and Process

- **Program (البرنامج):** A **passive entity** (كيان خامل) stored on secondary storage (disk) as an executable file.
- **Process (العملية):** A **program in execution** (برنامج قيد التنفيذ). It is an **active entity** (كيان نشط) and represents a unit of work within the operating system.

Process Control Block - PCB

- A data structure that holds all information required to execute a process successfully.
- **Most Important Information:** The **Process State** (حالة العملية الحالية).

2. Process States & Models

Process States

1. **New:** The process is being created (جاري إنشاء العملية).
2. **Running:** Instructions are actively being executed by the CPU (المعالج ينفذ تعليمات العملية حالياً).
3. **Waiting / Blocked :** The process is waiting for an event, such as I/O completion or signal (العملية تنتظر حدثاً معيناً مثل مدخلات/مخرجات).
4. **Ready:** The process is loaded into main memory and waiting to be assigned to the CPU (جاهزة في الذاكرة وتنتظر دورها على المعالج).
5. **Terminated :** The process has finished its execution (انتهت العملية تماماً).

Process Models

- **2-State Model:** Simple transition between *Not Running* and *Running*.
- **3-State Model:** Transitions between *Ready*, *Running*, and *Blocked / Waiting*.
- **5-State Model:** Full life-cycle including *New*, *Ready*, *Running*, *Waiting*, and *Terminated*.

3. Process Execution & CPU–I/O Burst Cycle

CPU–I/O Burst Cycle

- Multiprogramming achieves maximum CPU utilization by alternating CPU execution and I/O wait cycles.
- Process execution consists of alternating **CPU Bursts** (computation) and **I/O Bursts** (waiting for I/O).
- **Burst Cycle Sequence Example:**
    - **CPU Burst:** load, store, add, store, read from file
    - **I/O Burst:** wait for I/O
    - **CPU Burst:** store, increment, index, write to file
    - **I/O Burst:** wait for I/O
- The **CPU burst distribution** is of primary concern in CPU scheduling system design.
- **Histogram of CPU-burst durations:** Shows a large number of short CPU bursts and a small number of long CPU bursts across typical workloads.

4. Queues, Schedulers, and Dispatcher

Scheduling Queues

Processes migrate among various queues throughout their lifecycle:

- **Job Queue:** Set of all processes in the system.
- **Ready Queue:** Set of all processes residing in main memory that are ready and waiting to execute.
- **Device Queues:** Set of processes waiting for a specific I/O device.

Types of Schedulers

1. **Long-Term Scheduler (Job Scheduler):**
    - Selects processes from disk and loads them into main memory (Ready Queue).
    - Invoked infrequently (seconds, minutes) → can afford to be slow.
    - Controls the **degree of multiprogramming** (number of processes in memory).
2. **Short-Term Scheduler (CPU Scheduler):**
    - Selects among ready processes in memory and allocates the CPU to one of them.
    - Invoked very frequently (milliseconds) → must be extremely fast.
    - Sometimes the only scheduler present in simple operating systems.
3. **Medium-Term Scheduler:**
    - Used when the degree of multiprogramming needs to be decreased.
    - Implements **Swapping**: Removes processes from memory (paged out to disk) and brings them back later to resume execution.

Process Categories

- **I/O-Bound Process:** Spends more time doing I/O than computations; characterized by many short CPU bursts.
- **CPU-Bound Process:** Spends more time doing computations; characterized by a few very long CPU bursts.

The Dispatcher

- Module that gives control of the CPU to the process selected by the short-term scheduler.
- **Key Tasks:**
    1. Switching context.
    2. Switching to user mode.
    3. Jumping to the proper location in the user program to restart execution.
- **Dispatch Latency:** The time required for the dispatcher to stop one process and start running another.

5. Scheduling Types, Criteria, and Formulas

Scheduling Types

- **Non-Preemptive Scheduling (غير قابلة للاقتطاع) :** Once a CPU is allocated to a process, it keeps the CPU until it terminates or switches to a waiting state voluntarily.
- **Preemptive Scheduling (قابلة للاقتطاع) :** A process can be interrupted and moved from running state to ready state (or waiting to ready state) by the OS kernel.

Scheduling Criteria

To achieve optimal system performance:

- **CPU Utilization (Maximize):** Keep the CPU as busy as possible
- **Throughput (Maximize):** Number of processes completed per unit of time.
- **Turnaround Time (Minimize):** Total time taken from process submission to its completion.
- **Waiting Time (Minimize):** Total time spent waiting in the ready queue.
- **Response Time (Minimize):** Time from request submission until the first response is produced (crucial for interactive/time-sharing environments).

Core Performance Formulas

1. **Turnaround Time (T.A.T):**

$$T.A.T = CompletionTime(C.T) - ArrivalTime(A.T)$$

2. **Average Turnaround Time (Avg. T.A.T):**

$$Avg.T.A.T = \frac{\sum T.A.T}{TotalNumberofProcesses}$$

3. **Waiting Time (W.T):**

$$W.T = T.A.T - BurstTime(B.T)$$

4. **Average Waiting Time (Avg. W.T):**

$$Avg.W.T = \frac{\sum W.T}{TotalNumberofProcesses}$$

5. **Throughput:**

$$Throughput = \frac{TotalNumberofProcesses}{TotalExecutionTimeforAllProcesses}$$

6. CPU Scheduling Policies List

1. **First-Come, First-Served (FCFS)**
2. **Shortest Job First (SJF)**
3. **Shortest Remaining Time First (SRTF)**
4. **Priority Scheduling**
5. **Round Robin (RR)**
6. **Multilevel Queues**
7. **Multilevel Feedback Queues**

7. First-Come, First-Served (FCFS) – Detailed Rules & Worked Examples

FCFS Core Rules

- **Execution Rule:** The job that arrives in the ready queue first is executed first by the CPU.
- **Default Arrival Time:** If arrival times are omitted, all processes are assumed to arrive at time $A.T = 0$.
- **Tie-Breaking Rule:** If multiple processes arrive at the exact same time, ties are broken using process IDs (e.g., P_1 before P_2).
- **Preemption Type: Non-preemptive**. The CPU will not be de-allocated from a process until its burst completes.

Example 01: FCFS (Arrival Order: not provided)

Inputs

- Processes: P_1 (B.T = 24), P_2 (B.T = 3), P_3 (B.T = 3)
- All Arrival Times ($A.T$) = 0.

Calculations Table

Process	Burst Time (B.T)	Arrival Time (A.T)	Completion Time (C.T)	Turnaround Time (T.A.T)	Waiting Time (W.T)
P1	24	0	24	$24 - 0 = 24$	$24 - 24 = 0$
P2	3	0	27	$27 - 0 = 27$	$27 - 3 = 24$
P3	3	0	30	$30 - 0 = 30$	$30 - 3 = 27$

Summary Metrics

- **Total Time:** 30 time units
- **Average Turnaround Time:** $\frac{24+27+30}{3} = \frac{81}{3} = 27$
- **Average Waiting Time:** $\frac{0+24+27}{3} = \frac{51}{3} = 17$
- **Throughput:** $\frac{3processes}{30units} = 0.1processes/unittime$

Example 02: FCFS (Arrival Order: P_2, P_3, P_1)

Inputs

- Processes: P_1 (B.T = 24), P_2 (B.T = 3), P_3 (B.T = 3)
- $A.T = 0$

Calculations Table

Process	Burst Time (B.T)	Arrival Time (A.T)	Completion Time (C.T)	Turnaround Time (T.A.T)	Waiting Time (W.T)
P1	24	0	30	$30 - 0 = 30$	$30 - 24 = 6$
P2	3	0	3	$3 - 0 = 3$	$3 - 3 = 0$
P3	3	0	6	$6 - 0 = 6$	$6 - 3 = 3$

Summary Metrics

- **Total Time:** 30 time units
- **Average Turnaround Time:** $\frac{30+3+6}{3} = \frac{39}{3} = 13$
- **Average Waiting Time:** $\frac{6+0+3}{3} = \frac{9}{3} = 3$
- **Throughput:** $\frac{3processes}{30units} = 0.1processes/unittime$

Example 03: FCFS Arrival Times

Inputs

- P_1: B.T = 24, A.T = 5
- P_2: B.T = 3, A.T = 1
- P_3: B.T = 3, A.T = 8

Arrival Sequence Timeline

1. At $t = 1$, P_2 arrives first → executes from $t = 1$ to $t = 4$.
2. At $t = 5$, P_1 arrives next → executes from $t = 5$ to $t = 29$.
3. At $t = 8$, P_3 arrives → waits in Ready Queue until P_1 finishes → executes from $t = 29$ to $t = 32$.

Calculations Table

Process	Burst Time (B.T)	Arrival Time (A.T)	Completion Time (C.T)	Turnaround Time (T.A.T)	Waiting Time (W.T)
P1	24	5	29	$29 - 5 = 24$	$24 - 24 = 0$
P2	3	1	4	$4 - 1 = 3$	$3 - 3 = 0$
P3	3	8	32	$32 - 8 = 24$	$24 - 3 = 21$

Summary Metrics

- **Total Time:** 32 time units
- **Average Turnaround Time:** $\frac{24+3+24}{3} = \frac{51}{3} = 17$
- **Average Waiting Time:** $\frac{0+0+21}{3} = \frac{21}{3} = 7$
- **Throughput:** $\frac{3processes}{32units} \approx 0.09 processes/unittime$

Example 04: FCFS (5-Process Dataset)

Inputs

Process	Arrival Time (A.T)	Burst Time (B.T)
P0	2	2
P1	1	3
P2	8	14
P3	4	2
P4	11	5

Chronological Arrival & Execution Flow

1. $t = 0$: CPU is idle.
2. $t = 1$: P_1 arrives first → scheduled ($t = 1 \rightarrow 4$).
3. $t = 2$: P_0 arrives → enters Ready Queue.
4. $t = 4$: P_1 completes. P_0 selected → scheduled ($t = 4 \rightarrow 6$).
5. $t = 4$: P_3 arrives → enters Ready Queue behind P_0.
6. $t = 6$: P_0 completes. P_3 selected → scheduled ($t = 6 \rightarrow 8$).
7. $t = 8$: P_3 completes. P_2 arrives → scheduled ($t = 8 \rightarrow 22$).
8. $t = 11$: P_4 arrives → waits in Ready Queue.
9. $t = 22$: P_2 completes. P_4 selected → scheduled ($t = 22 \rightarrow 27$).

Process	Burst Time (B.T)	Arrival Time (A.T)	Completion Time (C.T)	Turnaround Time (T.A.T)	Waiting Time (W.T)
P0	2	2	6	$6 - 2 = 4$	$4 - 2 = 2$
P1	3	1	4	$4 - 1 = 3$	$3 - 3 = 0$
P2	14	8	22	$22 - 8 = 14$	$14 - 14 = 0$
P3	2	4	8	$8 - 4 = 4$	$4 - 2 = 2$
P4	5	11	27	$27 - 11 = 16$	$16 - 5 = 11$

Summary Metrics

- **Total Execution Time:** 27 time units
- **Average Turnaround Time:** $\frac{4+3+14+4+16}{5} = \frac{41}{5} = 8.2$
- **Average Waiting Time:** $\frac{2+0+0+2+11}{5} = \frac{15}{5} = 3.0$
- **Throughput :** 0.19

FCFS – Summary

- Advantages
  - Easy and straightforward to implement
  - No starvation
- Disadvantages
  - Shorter burst time process had to wait behind long processes
  - Convoy Effect
- Solution
  - Shortest Job First (SJF)

Chapter 5

1. SJF Introduction & Core Rules

Key Rules of Non-Preemptive SJF :

1. **Selection Rule (قاعدة الاختيار):** The process with the **smallest CPU burst time** in the ready queue is assigned to the CPU first (العملية ذات وقت التنفيذ الأقصر يتم اختيارها أولاً).
2. **Default Arrival Time:** If Arrival Time ($A.T$) is not specified, assume $A.T = 0$ for all processes.
3. **Tie-Breaking Rule 1 — Arrival Time:** If two processes have the same burst time, the process that arrived earlier is executed first.
4. **Tie-Breaking Rule 2 — Process Identifier:** If both arrival time and burst time are identical, execution order defaults to the Process Identifier / ID (P_1 before P_2).
5. **Non-Preemptive Behavior (غير قابلة للاقتطاع):** Once a process gains control of the CPU, it runs to completion without interruption.

2.Examples

Example 01:

Inputs

- P_1: $A.T = 0, B.T = 24$
- P_2: $A.T = 0, B.T = 3$
- P_3: $A.T = 0, B.T = 3$

Since all processes arrive at $t = 0$, SJF selects the shortest burst time first:

1. P_2 $(B.T = 3)$ and P_3 $(B.T = 3)$ are tied. P_2 runs first by ID $(0 \rightarrow 3)$.
2. P_3 $(B.T = 3)$ runs next $(3 \rightarrow 6)$.
3. P_1 $(B.T = 24)$ runs last $(6 \rightarrow 30)$.

Process Calculation Table

Process	Burst Time (B.T)	Arrival Time (A.T)	Completion Time (C.T)	Turnaround Time (T.A.T = C.T - A.T)	Waiting Time (W.T = T.A.T - B.T)
P1	24	0	30	$30 - 0 = 30$	$30 - 24 = 6$
P2	3	0	3	$3 - 0 = 3$	$3 - 3 = 0$
P3	3	0	6	$6 - 0 = 6$	$6 - 3 = 3$

Summary Metrics

- **Average Turnaround Time :** $\frac{30+3+6}{3} = \frac{39}{3} = 13$
- **Average Waiting Time :** $\frac{6+0+3}{3} = \frac{9}{3} = 3$
- **Throughput :** $\frac{3}{30} = 0.1 processes/unittime$

Example 02:

Inputs

- P_1: $A.T = 0, B.T = 24$
- P_2: $A.T = 5, B.T = 3$
- P_3: $A.T = 8, B.T = 3$

Execution Decision

1. At $t = 0$, only P_1 is present in the Ready Queue. P_1 starts running $(0 \rightarrow 24)$.
2. At $t = 5$, P_2 arrives. At $t = 8$, P_3 arrives.
3. At $t = 24$, P_1 finishes. Both P_2 $(B.T = 3)$ and P_3 $(B.T = 3)$ are waiting. Since P_2 arrived earlier ($t = 5$ vs $t = 8$), P_2 executes next $(24 \rightarrow 27)$.
4. P_3 executes last $(27 \rightarrow 30)$.

Process Calculation Table

Process	Burst Time (B.T)	Arrival Time (A.T)	Completion Time (C.T)	Turnaround Time (T.A.T)	Waiting Time (W.T)
P1	24	0	24	$24 - 0 = 24$	$24 - 24 = 0$
P2	3	5	27	$27 - 5 = 22$	$22 - 3 = 19$
P3	3	8	30	$30 - 8 = 22$	$22 - 3 = 19$

Summary Metrics

- **Average Turnaround Time:** $\frac{24+22+22}{3} = \frac{68}{3} \approx 22.7$
- **Average Waiting Time:** $\frac{0+19+19}{3} = \frac{38}{3} \approx 12.7$
- **Throughput:** $\frac{3}{30} = 0.1 processes/unittime$

Example 03:

Inputs

- P_1: $A.T = 5, B.T = 24$
- P_2: $A.T = 1, B.T = 3$
- P_3: $A.T = 8, B.T = 3$

Execution Timeline

1. $t = 0 \rightarrow 1$: CPU is idle (المعالج خامل).
2. $t = 1$: P_2 arrives → starts execution $(1 \rightarrow 4)$.
3. $t = 4 \rightarrow 5$: CPU is idle again.
4. $t = 5$: P_1 arrives → starts execution $(5 \rightarrow 29)$.
5. $t = 8$: P_3 arrives while P_1 is running → waits in queue.
6. $t = 29$: P_1 finishes → P_3 executes $(29 \rightarrow 32)$.

Process Calculation Table

Process	Burst Time (B.T)	Arrival Time (A.T)	Completion Time (C.T)	Turnaround Time (T.A.T)	Waiting Time (W.T)
P1	24	5	29	$29 - 5 = 24$	$24 - 24 = 0$
P2	3	1	4	$4 - 1 = 3$	$3 - 3 = 0$
P3	3	8	32	$32 - 8 = 24$	$24 - 3 = 21$

Summary Metrics

- **Average Turnaround Time:** $\frac{24+3+24}{3} = \frac{51}{3} = 17$
- **Average Waiting Time:** $\frac{0+0+21}{3} = \frac{21}{3} = 7$
- **Throughput:** $\frac{3}{32} \approx 0.09 processes/unittime$

Example 04:

Inputs

- P_1: $A.T = 0, B.T = 6$
- P_2: $A.T = 0, B.T = 8$
- P_3: $A.T = 0, B.T = 7$
- P_4: $A.T = 0, B.T = 3$

Execution Order by SJF

1. P_4 $(B.T = 3)$ → runs $0 \rightarrow 3$
2. P_1 $(B.T = 6)$ → runs $3 \rightarrow 9$
3. P_3 $(B.T = 7)$ → runs $9 \rightarrow 16$
4. P_2 $(B.T = 8)$ → runs $16 \rightarrow 24$

Process Calculation Table

Process	Burst Time (B.T)	Arrival Time (A.T)	Completion Time (C.T)	Turnaround Time (T.A.T)	Waiting Time (W.T)
P1	6	0	9	$9 - 0 = 9$	$9 - 6 = 3$
P2	8	0	24	$24 - 0 = 24$	$24 - 8 = 16$
P3	7	0	16	$16 - 0 = 16$	$16 - 7 = 9$
P4	3	0	3	$3 - 0 = 3$	$3 - 3 = 0$

Summary Metrics

- **Average Turnaround Time:** $\frac{9+24+16+3}{4} = \frac{52}{4} = 13$
- **Average Waiting Time:** $\frac{3+16+9+0}{4} = \frac{28}{4} = 7$
- **Throughput:** $\frac{4}{24} \approx 0.16 processes/unittime$

4. Student Exercises

Example 05: Priority-based Queue Entry with SJF

Rule for Queue Entry:

If multiple processes arrive at the **exact same time**, they enter the Ready Queue based on **Priority Number** (Higher number = Higher priority). Once in the Ready Queue, SJF rules apply.

Given Data

- P_0: $A.T = 3, B.T = 7, Priority = 2$
- P_1: $A.T = 5, B.T = 10, Priority = 6$
- P_2: $A.T = 10, B.T = 2, Priority = 4$
- P_3: $A.T = 6, B.T = 9, Priority = 3$
- P_4: $A.T = 1, B.T = 5, Priority = 1$
- P_5: $A.T = 4, B.T = 6, Priority = 5$

Step-by-Step Execution

1. $t = 0 \rightarrow 1$: Idle.
2. $t = 1$: P_4 arrives → starts running $(1 \rightarrow 6)$.
3. **During P_4's run ($t = 1 \rightarrow 6$):**
   - $t = 3$: P_0 arrives $(B.T = 7)$.
   - $t = 4$: P_5 arrives $(B.T = 6)$.
   - $t = 5$: P_1 arrives $(B.T = 10)$.
4. **At $t = 6$ (P_4 finishes):** Waiting in Ready Queue are P_0 (7), P_5 (6), P_1 (10). Shortest is P_5 $(B.T = 6)$.
   - P_5 runs $(6 \rightarrow 12)$.
   - $t = 6$: P_3 arrives $(B.T = 9)$.
   - $t = 10$: P_2 arrives $(B.T = 2)$.
5. **At $t = 12$ (P_5 finishes):** Waiting are P_0 (7), P_1 (10), P_3 (9), P_2 (2). Shortest is P_2 ($B.T = 2$).
   - P_2 runs $(12 \rightarrow 14)$.
6. **At $t = 14$ (P_2 finishes):** Waiting are P_0 (7), P_3 (9), P_1 (10). Shortest is P_0 $(B.T = 7)$.
   - P_0 runs $(14 \rightarrow 21)$.
7. **At $t = 21$ (P_0 finishes):** Shortest remaining is P_3 $(B.T = 9)$.
   - P_3 runs $(21 \rightarrow 30)$.
8. **At $t = 30$ (P_3 finishes):** Last process is P_1 $(B.T = 10)$.
   - P_1 runs $(30 \rightarrow 40)$.

Final Table

Process	Burst Time	Arrival Time	Completion Time	Turnaround Time (C.T−A.T)	Waiting Time (T.A.T−B.T)
P0	7	3	21	$21 - 3 = 18$	$18 - 7 = 11$
P1	10	5	40	$40 - 5 = 35$	$35 - 10 = 25$
P2	2	10	14	$14 - 10 = 4$	$4 - 2 = 2$
P3	9	6	30	$30 - 6 = 24$	$24 - 9 = 15$
P4	5	1	6	$6 - 1 = 5$	$5 - 5 = 0$
P5	6	4	12	$12 - 4 = 8$	$8 - 6 = 2$

- **Average Turnaround Time:** $\frac{18+35+4+24+5+8}{6} = \frac{94}{6} \approx 15.67$
- **Average Waiting Time:** $\frac{11+25+2+15+0+2}{6} = \frac{55}{6} \approx 9.17$
- **Throughput:** $\frac{6}{40} = 0.15 processes/unittime$

Example 06:

Given Data

- P_0: $A.T = 2, B.T = 2$
- P_1: $A.T = 1, B.T = 3$
- P_2: $A.T = 8, B.T = 14$
- P_3: $A.T = 4, B.T = 2$
- P_4: $A.T = 11, B.T = 5$

Step-by-Step Execution

1. $t = 0 \rightarrow 1$: Idle.
2. $t = 1$: P_1 arrives → runs $(1 \rightarrow 4)$.
3. **During P_1's run:** P_0 arrives at $t = 2$, P_3 arrives at $t = 4$.
4. **At $t = 4$:** Waiting are P_0 $(B.T = 2)$ and P_3 $(B.T = 2)$. Tied burst time → P_0 arrived earlier ($t = 2$ vs $t = 4$), so P_0 runs $(4 \rightarrow 6)$.
5. **At $t = 6$:** P_3 runs $(6 \rightarrow 8)$.
6. **At $t = 8$:** P_2 arrives $(B.T = 14)$ → runs $(8 \rightarrow 22)$.
7. **At $t = 11$:** P_4 arrives $(B.T = 5)$ while P_2 is running.
8. **At $t = 22$:** P_4 runs $(22 \rightarrow 27)$.

Process	Burst Time	Arrival Time	Completion Time	Turnaround Time	Waiting Time
P0	2	2	6	$6 - 2 = 4$	$4 - 2 = 2$
P1	3	1	4	$4 - 1 = 3$	$3 - 3 = 0$
P2	14	8	22	$22 - 8 = 14$	$14 - 14 = 0$
P3	2	4	8	$8 - 4 = 4$	$4 - 2 = 2$
P4	5	11	27	$27 - 11 = 16$	$16 - 5 = 11$

- **Average Turnaround Time:** $\frac{4+3+14+4+16}{5} = \frac{41}{5} = 8.2$
- **Average Waiting Time:** $\frac{2+0+0+2+11}{5} = \frac{15}{5} = 3.0$
- **Throughput:** $\frac{5}{27} \approx 0.185 processes/unittime$

5. Summary: Advantages, Disadvantages & Solution

Advantages (المميزات)

- **Optimal Average Waiting Time (مثالية في زمن الانتظار):** Gives the **minimum average waiting time** for any given set of processes.
- **No Convoy Effect (لا يوجد تأثير القافلة):** Short processes do not get stuck waiting behind long processes like in FCFS.

Disadvantages (العيوب)

- **Burst Time Prediction Friction (صعوبة معرفة زمن التنفيذ):** Requires knowing or estimating the length of the next CPU burst in advance, which is difficult in real operating systems.
- **Potential Starvation (احتمالية تجويع العمليات الطويلة):** Long processes may wait indefinitely if short processes keep arriving.

Solution (الحل)

- Use **Shortest Remaining Time First (SRTF)** — the **preemptive version** of SJF that dynamically re-evaluates remaining time whenever a new process arrives.